ParkirKu

Pencatat Parkir Kampus Surabaya - Tugas Individu #1 Algoritma dan Pemrograman (BBK1AAB4) 2026

Nama lengkap	Ahmad Rafid Zaidan
NIM	108052630010
Kelas	SS1SI-50-REG-05
Dosen pengampu	Anisa Dzulkarnain, S.Kom., M.Kom.

Daftar Isi

1. Identitas Diri
2. Deskripsi Program
3. Dekomposisi Masalah
4. Flowchart & Pseudocode Tahap 3
5. Tabel Penelusuran
6. Tabel Kasus Uji
7. Catatan Galat
8. Refleksi
9. Pengungkapan Penggunaan AI

2. Deskripsi Program

ParkirKu adalah program berbasis teks (console) yang dipakai petugas gerbang untuk mencatat kendaraan yang keluar dari area parkir kampus selama satu hari. Petugas memasukkan plat, jenis kendaraan (motor atau mobil), jam dan menit masuk serta keluar, status kartu civitas, dan status tiket. Jika ada isian yang keliru, program menolaknya dan meminta petugas mengisi ulang sampai benar. Setelah data valid, program menghitung durasi, jam ditagih, dan tarif dasar (dengan batas gratis 15 menit dan batas maksimum per hari). Program juga menghitung diskon 20% untuk kartu civitas, denda tiket hilang, serta total bayar, lalu mencetak struk. Pencatatan berlanjut sampai petugas mengetik `SELESAI`, dan baru setelah itu program menampilkan rekap harian lengkap dengan grafik batang teks.

Program terbagi menjadi lima tahap yang berurutan. Materi yang dipakai hanya dari pekan 1–6: variabel, tipe data, `input()`, operator, `if/elif/else`, `and/or/not`, `while`, `for`, dan `range()`. Saya tidak memakai `def`, `list`, `tuple`, `dictionary`, `import`, `try/except`, `break`, maupun `continue`. Seluruh angka aturan disimpan sebagai konstanta berhuruf kapital di bagian atas program, sehingga rumus tidak berisi “angka ajaib”. Selain fitur wajib, saya menambahkan beberapa fitur untuk memudahkan petugas:

- Plat kosong ditolak. Kata `SELESAI` dikenali tanpa memperhatikan huruf besar atau kecil, jadi `selesai` juga menghentikan program.
- Isian jam atau menit yang berisi huruf (misalnya `8a`) tidak membuat program galat, sebab `isdigit()` memeriksanya terlebih dahulu sebelum `int()` dipanggil, tanpa `try/except`.

- Jawaban jenis kendaraan dan ya/tidak boleh ditulis dengan huruf besar (`MOTOR`, `Ya`) dan boleh memuat spasi berlebih.
- Rekap memuat tambahan berupa pendapatan motor dan mobil serta total diskon yang diberikan. Grafik batangnya juga disertai angka jumlah.

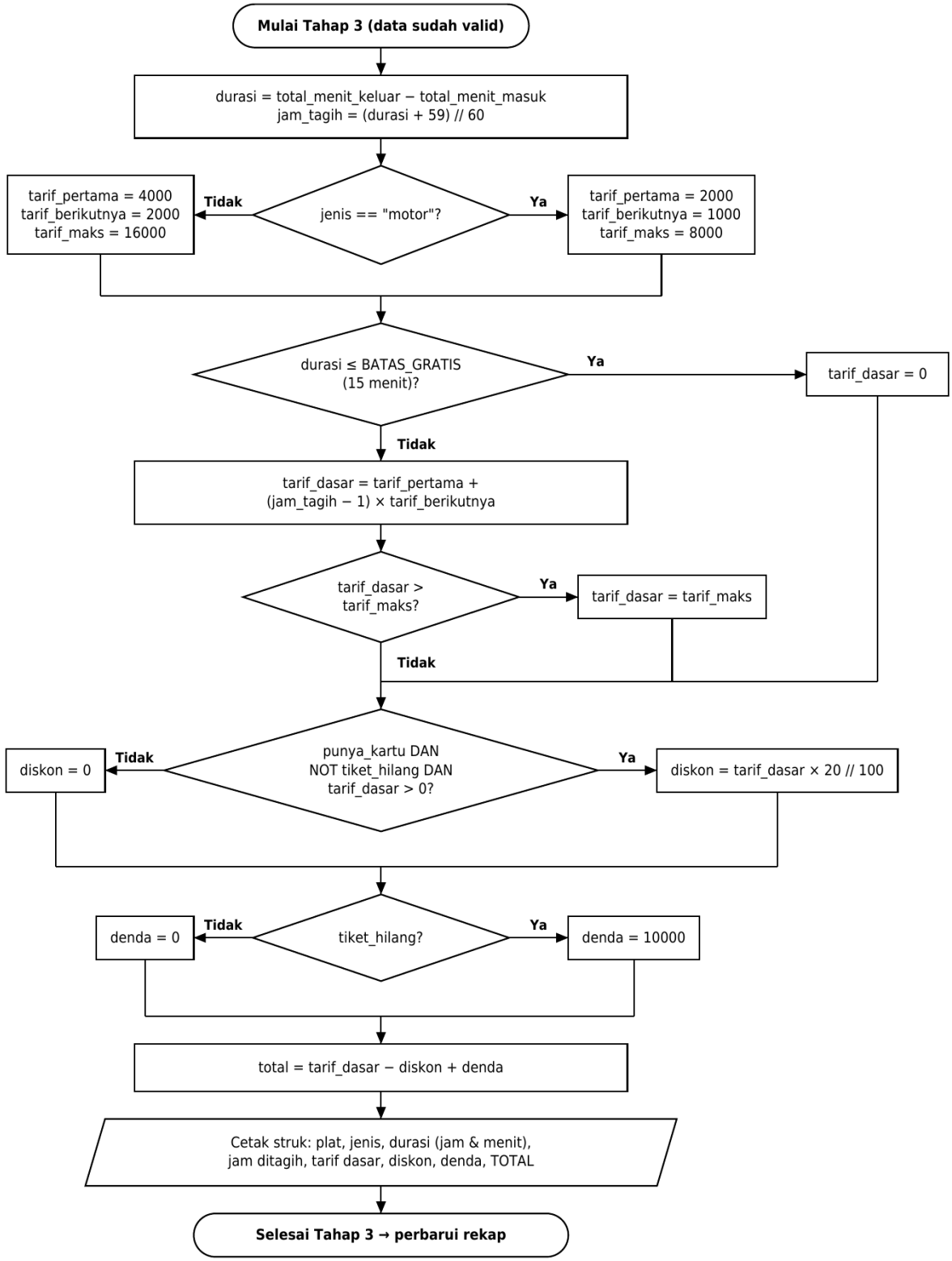
3. Dekomposisi Masalah

Masalah besar “mencatat parkir satu hari” saya pecah menjadi beberapa sub-masalah berikut, lengkap dengan rencana pemrogramannya.

No	Sub-masalah	Plan pemrograman
1	Mengumpulkan semua angka aturan tarif dan kata penanda dalam satu tempat	Konstanta bernama (huruf kapital)
2	Mencetak tabel tarif 1–8 jam dengan batas maksimum yang sudah diterapkan	`for` + `range()`, penjaga batas atas (capping)
3	Menetapkan nilai awal semua variabel rekap sebelum perulangan	Pencacah/akumulator diawali 0, ekstrem diawali −1
4	Mengulang pencatatan meski jumlah kendaraan tidak diketahui	Sentinel (priming read + baca ulang di akhir blok)
5	Menolak jenis kendaraan selain motor dan mobil	Validasi `while` + kondisi majemuk `and`
6	Menolak jam di luar 0–23 dan menit di luar 0–59	Validasi `while` + variabel penanda sah + `or`
7	Menolak waktu keluar yang mendahului waktu masuk	Penjaga rentang, `if/elif/else`
8	Menolak jawaban selain ya atau tidak (kartu dan tiket)	Validasi `while` + `and`
9	Menghitung durasi dan jam ditagih (pembulatan ke atas)	Konversi ke total menit, `//` dan `%`
10	Menentukan tarif menurut jenis kendaraan	`if/else`
11	Menentukan tarif dasar: gratis (≤ 15 menit), normal, atau berhenti di batas maksimum	`if/else` bersarang, penjaga batas
12	Memberi diskon 20% hanya jika punya kartu, tiket tidak hilang, dan tarif > 0	Kondisi majemuk `and` + `not`
13	Menghitung denda tiket hilang (tetap berlaku walau parkir gratis), total, dan struk	`if/else`, ekspresi aritmetika
14	Mencacah jumlah kendaraan, motor, dan mobil	Pencacah
15	Mencacah kendaraan yang gratis, berdiskon, dan kena denda	Penyaringan + pencacah
16	Total pendapatan (dan per jenis)	Akumulator
17	Parkir terlama dan bayar tertinggi (jika seri, yang lebih dulu dipertahankan)	Nilai ekstrem dengan > (bukan >=)
18	Menghitung rata-rata dan rekap tanpa pembagian dengan nol	Penjaga data kosong, akumulator ÷ pencacah (`//`)
19	Membuat grafik batang teks untuk motor dan mobil	`for` + `range()`, `print(end="")`
20	Membandingkan waktu masuk dan keluar dengan tepat (misalnya 10:30 dan 10:29, jam sama tetapi menit lebih kecil)	Ubah ke total menit lalu bandingkan satu angka
21	Mencegah galat ketika jam atau menit berisi huruf	Penjaga `isdigit()` sebelum `int()`
22	Plat kosong tidak boleh dicatat, dan `selesai` berhuruf kecil tetap berfungsi sebagai penanda	Validasi `while`, normalisasi `.strip().upper()`
23	Durasi 0 menit (masuk = keluar) harus dianggap sah dan gratis	Kondisi < (bukan <=) pada penjaga waktu keluar
24	Menetapkan arti “parkir gratis” ketika tiket hilang	Penyaringan berdasarkan `tarif_dasar == 0` (sesuai aturan 3.0)
25	Nilai awal ekstrem yang tetap benar walau semua durasi atau total bernilai 0	Nilai awal −1 (lebih kecil dari nilai sah terkecil)
26	Menampilkan durasi dalam jam dan menit	`durasi // 60` dan `durasi % 60`

4. Flowchart & Pseudocode Tahap 3 (Hitung Biaya)

Angka pada kotak flowchart ditulis sebagai nilai langsung agar mudah dibaca, sedangkan di kode semuanya berupa konstanta.



Pseudocode Tahap 3

```
MULAI TAHAP 3                       { semua masukan sudah valid dari Tahap 2 }
  durasi    <- total_menit_keluar - total_menit_masuk
  jam_tagih <- (durasi + 59) div 60     { pembulatan ke atas ke jam penuh }

  JIKA jenis = "motor" MAKA
      tarif_pertama <- TARIF_MOTOR_PERTAMA ; tarif_berikutnya <- TARIF_MOTOR_BERIKUTNYA
      tarif_maks    <- TARIF_MOTOR_MAKS
  SELAIN ITU
      tarif_pertama <- TARIF_MOBIL_PERTAMA ; tarif_berikutnya <- TARIF_MOBIL_BERIKUTNYA
      tarif_maks    <- TARIF_MOBIL_MAKS

  JIKA durasi <= BATAS_GRATIS_MENIT MAKA
      tarif_dasar <- 0
  SELAIN ITU
      tarif_dasar <- tarif_pertama + (jam_tagih - 1) * tarif_berikutnya
      JIKA tarif_dasar > tarif_maks MAKA tarif_dasar <- tarif_maks

  JIKA punya_kartu DAN BUKAN tiket_hilang DAN tarif_dasar > 0 MAKA
      diskon <- tarif_dasar * PERSEN_DISKON div 100
  SELAIN ITU
      diskon <- 0

  JIKA tiket_hilang MAKA denda <- DENDA_TIKET_HILANG SELAIN ITU denda <- 0

  total <- tarif_dasar - diskon + denda
  TAMPILKAN struk: plat, jenis, durasi (durasi div 60 jam, durasi mod 60 menit),
                   jam_tagih, tarif_dasar, diskon, denda, total
SELESAI TAHAP 3
```

Mengapa batas maksimum diterapkan setelah rumus dihitung?
Rumus yang sama persis dipakai pada tabel Tahap 1, sehingga angka di tabel pasti sama dengan angka yang ditagih. Urutan ini juga membuat kasus “tepat di batas” tetap aman: motor 7 jam = Rp 8000 tidak terpotong karena kondisinya memakai >, bukan >=.

5. Tabel Penelusuran (Data 5.1)

Saya menghitung penelusuran ini dengan tangan terlebih dahulu, kemudian membandingkan hasilnya dengan keluaran program.

5.1 Perhitungan tangan per kendaraan

Plat	Langkah perhitungan
A 1111 AA motor 13:00–13:40, kartu tidak, tiket tidak	masuk = 13×60+0 = 780; keluar = 13×60+40 = 820; durasi = 40 menit. jam_tagih = (40+59)//60 = 99//60 = **1**. Karena 40 > 15, tarif = 2000 + (1−1)×1000 = **2000** (≤ 8000). Tidak punya kartu, jadi diskon 0. Denda 0. Total = **2000**
B 2222 BB mobil 06:50–09:10, kartu ya, tiket tidak	masuk = 410; keluar = 550; durasi = **140** menit (2 jam 20 menit). jam_tagih = 199//60 = **3**. Tarif = 4000 + 2×2000 = **8000** (≤ 16000). Kartu ya, tiket tidak hilang, dan tarif > 0, maka diskon = 8000×20//100 = **1600**. Denda 0. Total = **6400**
C 3333 CC motor 15:00–15:15, kartu tidak, tiket tidak	masuk = 900; keluar = 915; durasi = **15** menit. jam_tagih = 74//60 = **1**. Karena 15 ≤ 15, tarif dasar = **0** (tepat di batas gratis). Diskon 0. Denda 0. Total = **0**

5.2 Tabel penelusuran variabel

Kendaraan ke-	`durasi`	`jam_tagih`	`tarif_dasar`	`diskon`	`total`	`jumlah`	`total_pendapatan`	`durasi_terlama`
Awal	-	-	-	-	-	0	0	-1
1 (A 1111 AA)	40	1	2000	0	2000	1	2000	40
2 (B 2222 BB)	140	3	8000	1600	6400	2	8400	140
3 (C 3333 CC)	15	1	0	0	0	3	8400	140

Variabel rekap lain pada akhir proses: motor 2, mobil 1, gratis 1 (C 3333 CC), diskon 1 (B 2222 BB), dan denda 0. Rata-rata = 8400 // 3 = 2800. Parkir terlama dimiliki B 2222 BB (2 jam 20 menit), dan bayar tertinggi juga B 2222 BB (Rp 6400). Pada kendaraan ke-3, `durasi_terlama` tetap 140 sebab 15 > 140 bernilai salah.

5.3 Perbandingan dengan hasil program

Besaran	Hitung tangan	Hasil program	Status
Total A 1111 AA / B 2222 BB / C 3333 CC	2000 / 6400 / 0	2000 / 6400 / 0	**Sama**
Jam ditagih (kendaraan 1 / 2 / 3)	1 / 3 / 1	1 / 3 / 1	**Sama**
Total pendapatan, rata-rata	8400, 2800	8400, 2800	**Sama**
Parkir terlama, bayar tertinggi	B 2222 BB (2j 20m), B 2222 BB (6400)	B 2222 BB (2j 20m), B 2222 BB (6400)	**Sama**

Mengapa nilai awal `durasi_terlama` adalah −1, bukan 0?
Pola nilai ekstrem membandingkan setiap data baru dengan "juara sementara" melalui `durasi > durasi_terlama`. Karena itu nilai awalnya harus lebih kecil daripada semua nilai yang sah, agar kendaraan pertama pasti menggantikannya. Durasi sah dimulai dari 0 menit, yaitu masuk dan keluar pada menit yang sama, misalnya 23:59–23:59. Jika nilai awalnya 0 dan kendaraan pertama berdurasi 0, maka `0 > 0` bernilai salah. Akibatnya `plat_terlama` tetap kosong dan rekap mencetak plat kosong. Dengan nilai awal −1, perbandingan `0 > -1` bernilai benar sehingga kendaraan pertama selalu tercatat. Alasan yang sama berlaku untuk `bayar_tertinggi = -1`, karena total bayar juga bisa Rp 0. Kasus ini sudah diuji (kasus uji B6). Operator > (bukan >=) juga memastikan kendaraan yang lebih dulu tercatat tetap dipertahankan ketika ada yang seri.

6. Tabel Kasus Uji

Semua kasus dijalankan pada program akhir, dan kolom "Hasil program" berisi keluaran yang sebenarnya. Kategorinya: N = normal, B = batas, T = tidak valid.

ID	Masukan	Hasil yang diharapkan	Hasil program
N1	L 1234 AB, motor, 08:00–11:20, kartu ya, tiket tidak	Durasi 3j 20m, jam 4, tarif 5000, diskon 1000, total Rp 4000	**Total Rp 4000**
N2	N 7788 IJ, mobil, 12:00–14:10, tidak, tidak	Jam 3, tarif 8000, total Rp 8000	**Total Rp 8000**
N3	A 1111 AA, motor, 13:00–13:40, tidak, tidak	Jam 1, total Rp 2000	**Total Rp 2000**
N4	Kelima data uji contoh (3.7), lalu `SELESAI`	5 kendaraan (3 motor, 2 mobil), 1 gratis, 2 diskon, 1 denda, Rp 30400, rata Rp 6080, terlama S 9012 EF, tertinggi L 3456 GH	**Semua sama persis**
B1	W 5678 CD, mobil, 09:00–09:15 (tepat 15 menit)	Tarif dasar 0, total Rp 0, terhitung gratis	**Total Rp 0, gratis 1**
B2	Motor 00:00–00:16 (16 menit, 1 menit lewat batas gratis), kartu ya	Jam 1, tarif 2000, diskon 400, total Rp 1600	**Total Rp 1600**
B3	Motor 10:00–11:00 (tepat 60 menit) lalu motor 10:00–11:01 (61 menit)	60 menit menjadi jam 1, Rp 2000; 61 menit menjadi jam 2, Rp 3000	**Rp 2000 dan Rp 3000**
B4	Motor 08:00–15:00 (7 jam, tepat Rp 8000 = maksimum)	Tarif 8000, tidak terpotong	**Tarif Rp 8000**
B5	S 9012 EF motor 07:30–19:45, kartu ya (ditagih 13 jam)	Rumus 14000 dipotong ke 8000, diskon 1600, total Rp 6400	**Total Rp 6400**
B6	Mobil 23:59–23:59 (durasi 0), kartu ya, tiket hilang	Jam 0, tarif 0, tanpa diskon, denda 10000, total Rp 10000; terlama tercatat (0 jam 0 menit)	**Total Rp 10000, terlama X 1**
B7	Mobil 00:00–23:59 (durasi maksimum sehari)	Jam 24, tarif dipotong 16000	**Total Rp 16000**

ID	Masukan	Hasil yang diharapkan	Hasil program
B8	Dua motor masing-masing 1 jam, Rp 2000 (seri)	Terlama & tertinggi = kendaraan pertama	**P1 untuk keduanya**
B9	Langsung `SELESAI` (data kosong)	"Belum ada kendaraan yang tercatat.", tanpa rata-rata/ekstrem, tidak galat	**Sesuai, tidak galat**
T1	Jenis sepeda	"Jenis harus motor atau mobil.", tanya ulang	**Sesuai**
T2	Jam masuk 25, menit 0; lalu jam −1; lalu menit 60	"Waktu tidak valid." / pesan angka, tanya ulang jam & menit	**Sesuai**
T3	Masuk 08:00, keluar 07:00	"Waktu keluar tidak boleh sebelum waktu masuk."	**Sesuai**
T4	Masuk 10:30, keluar 10:29 (jam sama, menit lebih kecil)	Ditolak; setelah diganti 10:45, durasi 15 menit dan gratis	**Ditolak, lalu Rp 0**
T5	Jam masuk `abc`; kartu `mungkin`; jenis `MOTOR`, jawaban `Ya` / `TIDAK`	Huruf ditolak tanpa galat; huruf besar diterima	**Sesuai, tidak galat**
T6	Plat kosong / hanya spasi, lalu `selesai`	"Plat nomor tidak boleh kosong.", lalu berhenti	**Sesuai**

7. Catatan Galat

Galat berikut muncul saat program dikembangkan dan diuji. Pesan galatnya saya salin langsung dari keluaran Python yang sebenarnya.

Jenis	Gejala	Cara menemukan	Perbaikan
Sintaks	Program sama sekali tidak mau berjalan dan menampilkan `SyntaxError: expected ':'` pada `if tiket_hilang`.	Galat sintaks terdeteksi sebelum baris pertama dijalankan. Python menunjuk baris dan posisi yang salah dengan tanda ^.	Menambahkan titik dua: `if tiket_hilang:`.
Runtime	Program berjalan tetapi berhenti di tengah dengan `ZeroDivisionError: integer division or modulo by zero` saat petugas langsung mengetik `SELESAI`.	Galat hanya muncul pada masukan tertentu (data kosong), dan traceback menunjuk baris `total_pendapatan // jumlah_kendaraan`.	Perhitungan rata-rata dipindahkan ke cabang `else` dari penjaga `if jumlah_kendaraan == 0`.
Runtime (dicegah)	`int("8a")` menghasilkan `ValueError: invalid literal for int()`.	Mengetik huruf pada isian jam sebagai percobaan.	Penjaga `isdigit()` sebelum `int()` (tanpa `try/except`).
Logika	Program berjalan tanpa pesan galat, tetapi angkanya keliru: L 1234 AB menjadi Rp 3200 (seharusnya 4000), L 3456 GH Rp 11000 (seharusnya 12000), dan total Rp 26600 (seharusnya 30400).	Saya membandingkan hasilnya dengan data uji 3.7 dan tabel penelusuran tangan. Struk menampilkan "Jam ditagih : 0" untuk durasi 16 menit, sehingga rumus menghasilkan 2000 + (0−1)×1000 = 1000.	Jam ditagih harus dibulatkan ke atas dengan `(durasi + 59) // 60`, bukan `durasi // 60`.

Cara membedakan tiga jenis galat
Galat **sintaks** menghentikan program sebelum sempat berjalan karena aturan penulisan dilanggar. Galat **runtime** muncul ketika program sedang berjalan, disertai traceback, karena ada operasi yang tidak mungkin dilakukan pada nilai tertentu. Galat **logika** paling sulit dikenali: tidak ada pesan apa pun dan program selesai seperti biasa, tetapi hasilnya salah. Satu-satunya cara menemukannya adalah membandingkan dengan hasil yang diharapkan (tabel penelusuran dan kasus uji).

8. Refleksi

Hal yang paling saya kuasai sejauh ini adalah pola validasi dengan `while` dan perulangan berpenanda. Kondisi `while` sudah cukup menunjukkan kapan perulangan berhenti, jadi saya tidak membutuhkan `while True` maupun `break`. Saya juga baru menyadari betapa pentingnya nilai awal pada pola nilai ekstrem: satu angka saja (0 atau −1) bisa membuat rekap salah tanpa pesan galat apa pun. Yang masih sulit bagi saya adalah menyusun kondisi majemuk agar tidak terbalik, misalnya kapan memakai `and` dan kapan `or` pada validasi "bukan motor dan bukan mobil". Saya juga perlu lebih banyak latihan untuk menemukan galat logika dengan cepat, karena galat jenis ini tidak memberi petunjuk selain angka yang keliru.

9. Pengungkapan Penggunaan AI

Pada tugas ini saya menggunakan asisten AI **Claude (Anthropic)**. AI membantu mengoreksi hasil dari kode `parkir.py`, flowchart dan README.

Saran yang saya pakai	Saran / alternatif yang saya tolak dan alasannya
• Membandingkan waktu dalam bentuk total menit, bukan jam dan menit secara terpisah. • Variabel penanda `waktu_masuk_sah` untuk memvalidasi beberapa kondisi sekaligus. • Penjaga `isdigit()` agar huruf pada isian angka tidak membuat program galat. • Nilai awal −1 untuk variabel ekstrem dan operator > untuk kasus seri. • Membatasi tarif pada nilai maksimum dengan rumus yang sama seperti tabel Tahap 1.	• `while True` + `break` untuk perulangan: dilarang oleh soal, dan kondisinya jadi tidak terlihat di baris `while`. • `try/except ValueError` untuk masukan huruf: dilarang oleh soal, sehingga diganti dengan `isdigit()`. • `print("#" * jumlah)` untuk grafik: soal mewajibkan `for` dan `range()`. • Menyimpan semua kendaraan ke list lalu mencari maksimum: list dilarang, dan memang tidak perlu karena nilai ekstrem bisa diperbarui langsung pada setiap kendaraan. • Menulis rumus tarif dua kali di `elif`: diganti dengan pola hitung-lalu-potong agar sama dengan tabel.